

Paterni ponašanja u sistemu za upravljanje klinikom

Patern	Gdje se primjenjuje	Razlog uvođenja
State	Klasa Termin / StatusTermina	Kontrola dozvoljenih prijelaza između stanja termina
Observer	Promjena statusa termina	Automatsko obavješćavanje zainteresovanih komponenti
Strategy	SistemZaSkeniranjQRKoda	Zamjenjivi algoritmi validacije QR koda
Command	zakaziTermin, otkaziTermin	Historijat akcija i podrška za poništavanje operacija
Chain of Responsibility	provjeriDostupnost	Strukturirani lanac provjera dostupnosti termina

1 State patern – upravljanje statusom termina

State patern omogućava objektu da mijenja svoje ponašanje u zavisnosti od trenutnog stanja. U kontekstu ovog sistema, klasa Termin prolazi kroz definisani životni ciklus stanja.

Prijedlog implementacije podrazumijeva uvođenje apstraktne klase IStatusTermina sa metodama kao što su zakaziTermin(), otkaziTermin() i evidentiraDolazak(). Svaki konkretan status (ZakazanStatus, OtkazanStatus, RealizovanStatus, PacijentPrisutanStatus) implementirao bi ovu klasu i definisao koja su stanja dozvoljena iz tog stanja.

Na primjer, iz stanja 'zakazan' moguće je prijeći u 'otkazan' ili 'realizovan', ali ne direktno u 'pacijentPrisutan'. Ovaj patern osigurava da logika tranzicija bude centralizovana i konzistentna.

2 Observer patern – obavješćavanje o promjeni stanja

Observer patern definiše zavisnost jedan-prema-više između objekata, tako da kada jedan objekat promijeni stanje, svi zavisni objekti automatski budu obavješćeni. U ovom sistemu, klasa Termin bila bi 'subjekt' (publisher), a različiti dijelovi sistema bili bi 'posmatrači' (subscribers).

Konkretni posmatrači mogli bi biti: NotifikacioniServis (slanje potvrde pacijentu), DnevnikAktivnosti (bilježenje promjene), i RasporedServis (oslobađanje termina pri otkazivanju). Uvođenjem ovog patterna izbjegla bi se direktna čvrsta sprega između klase Termin i ostalih servisa.

3 Strategy patern – validacija QR koda

Strategy patern omogućava definisanje familije algoritama, enkapsulaciju svakog od njih i njihovu međusobnu zamjenjivost. Klasa SistemZaSkeniranjQRKoda trenutno implementira jednu konkretnu logiku validacije.

Uvođenjem ovog patterna, definisao bi se interfejs IStrategijaValidacije sa metodom validiraj(vrijednostKoda). Konkretno implementacije bile bi npr. OnlineValidacija,

OfflineValidacija, ili ValidacijaSalsticanjem. Klasa sistema za skeniranje primala bi željenu strategiju kroz konstruktor ili setter, što bi omogućilo promjenu načina validacije bez izmjene same klase.

4 Command patern – operacije nad terminima

Command patern enkapsulira zahtjev kao objekat, što omogućava pohranjivanje, redanje i poništavanje operacija. Metode zakaziTermin, otkaziTermin i izmijeniTermin u TerminController-u pogodne su kandidate za ovaj patern.

Svaka operacija bila bi predstavljena kao zaseban objekat komande koji implementira interfejs IKomanda sa metodama izvrsi() i ponistilzrsavanje(). Upravnik komandama (CommandManager) čuvao bi historijat izvršenih komandi, što otvara mogućnost audit loga i opcionalnog mehanizma poništavanja.

5 Chain of Responsibility – provjera dostupnosti

Chain of Responsibility patern prosleđuje zahtjev kroz lanac handlera sve dok jedan od njih ne obradi zahtjev. Metoda provjeriDostupnost u Raspored klasi vrši niz provjera koje su trenutno implementirane kao sekvencijalni if-else blokovi.

Uvođenjem ovog patterna, svaka provjera bila bi zaseban handler: ProvjeraRasporedaDoktora, ProvjeraDostupnostiTermina, ProvjeraKapaciteta, itd. Handleri bi bili poredani u lanac i svaki bi ili obradio zahtjev ili ga proslijedio sljedećem. Ovo olakšava dodavanje novih provjera bez izmjene postojeće logike.